

Laboratory Report: Real-Time Multisensor Room Monitoring System

Course: BCA152 Microcontrollers — Laboratory Activity No. 1

Student Name: Gazawe Casim

Instructor: Asst. Prof. Paul Rodolf P. Castor, M.Sc.

Institution: Department of Computer Applications, College of Computer Studies, MSU-IIT

1. Introduction

A real-time environmental room monitoring system was designed, implemented, and verified on the ESP32 microcontroller using the Espressif IoT Development Framework (ESP-IDF) and native FreeRTOS concurrency primitives within the Wokwi simulation environment.

The objective of the activity was to transition away from traditional single-loop architectures by decomposing system requirements into six concurrent FreeRTOS tasks. The system monitors ambient temperature and relative humidity via a DHT22 sensor, relative light intensity through a photoresistor (LDR) analog input, and human room presence through a passive infrared (PIR) sensor. Data is rendered onto an SSD1306 128×64 OLED display over I2C and navigated using a rotary encoder dial. Out-of-bound temperature conditions trigger an audible buzzer alarm, while prolonged inactivity initiates an automated power-saving sleep state.

2. Design and Testing

A. General Requirements & Methodology

- **Concept:** A monolithic super-loop with blocking delays introduces latency that drops mechanical rotary encoder pulses and produces execution drift during 1-wire sensor

handshakes. A preemptive multitasking architecture was implemented to decouple time-critical input sampling from slower display rendering and periodic sensor acquisition.

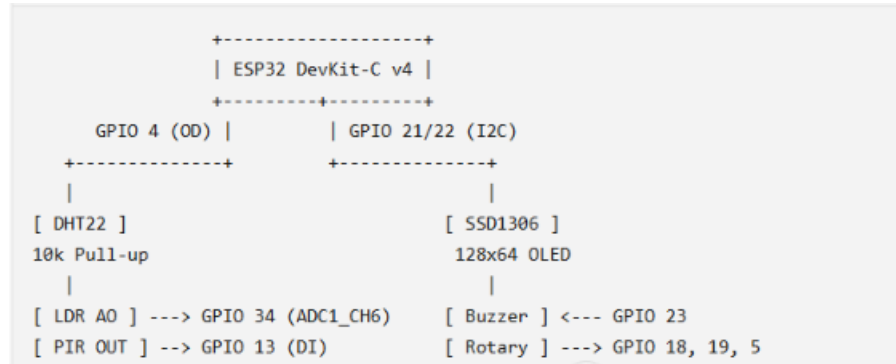
- **Implementation:** The application was constructed using PlatformIO with framework = espidf. Arduino abstractions were excluded. The firmware entry point is app_main(). Inter-task communication was implemented using dedicated FreeRTOS queues to prevent queue stealing, a binary mutex to synchronize UART stdout, and an event group for system-wide flags.
- **Testing Approach:** Pure decision logic was separated from hardware drivers into deterministic functions (evaluateTemperature, nextDisplayMode, previousDisplayMode, and evaluateSystemState). These were validated on the host machine using automated unit tests under the Unity framework before full integration into the simulated circuit.

B. Hardware Description & Pinout

Component	Pin Function	ESP32 GPIO	Mode/Protocol	Hardware Notes
DHT22	SDA (DATA)	GPIO 4	Open-Drain Bidirectional	10 kΩ pull-up to 3.3V bus
Photoresistor (LDR)	AO (Analog Out)	GPIO 34	ADC1 Channel 6	12-bit conversion (0–4095)
PIR Sensor	OUT (Signal)	GPIO 13	Digital Input	Internal pull-down enabled
Rotary Encoder	CLK (Phase A)	GPIO 18	Digital Input	Internal pull-up enabled
Rotary Encoder	DT (Phase B)	GPIO 19	Digital Input	Internal pull-up enabled
Rotary Encoder	SW (Button)	GPIO 5	Digital Input	Momentary active-low reset
SSD1306 OLED	SDA	GPIO 21	I2C Master Data	400 kHz clock rate
SSD1306 OLED	SCL	GPIO 22	I2C Master Clock	400 kHz clock rate

Buzzer	Signal (+)	GPIO 23	Digital Output	Active-high push-pull
--------	------------	---------	----------------	-----------------------

All components share a common 3.3V rail and GND bus supplied by the ESP32 board.



C. Software Description & Concurrency Architecture

I. Task Decomposition

The firmware executes across six cooperating tasks:

1. **InputTask (Priority 3, Core 1):** Polls rotary encoder CLK and DT pins at a 10 ms interval. Detects falling edges and determines rotation direction, updating the active DisplayMode with bidirectional wraparound.
2. **MotionTask (Priority 3, Core 1):** Samples the PIR digital output every 100 ms. Sets EVENT_MOTION and EVENT_ACTIVE bits in xSystemEvents upon detected motion, and clears EVENT_MOTION when room motion ceases.
3. **SensorTask (Priority 2, Core 0):** Executes strictly every 2000 ms using vTaskDelayUntil(). Acquires temperature and humidity via bit-banged 1-wire protocol in a critical section, reads relative light via the ESP-IDF ADC oneshot driver, packages data into a SensorData struct, and publishes it to consumer queues.
4. **AlarmTask (Priority 2, Core 0):** Blocks on xSensorQueueAlarm. When a new sample arrives, it calls evaluateTemperature(). If temperature is < 18.0 °C or > 30.0 °C, GPIO 23 is driven HIGH, the buzzer sounds, and EVENT_ALARM is set. Otherwise, GPIO 23 is driven LOW.

5. **StateTask (Priority 2, Core 0):** Executes on a 1-second periodic tick. Increments an inactivity counter when no motion is detected. When elapsed inactivity reaches 15 seconds, the system transitions to INACTIVE and clears EVENT_ACTIVE. Motion detection resets the counter and restores ACTIVE.
6. **DisplayTask (Priority 1, Core 1):** Exclusively owns the SSD1306 OLED. Blocks on xSensorQueueDisplay with a 300 ms timeout. When EVENT_ACTIVE is cleared, it renders a low-power "SYSTEM SLEEP" banner. When active, it renders the header, the selected measurement page, and dynamic status/alarm banners.

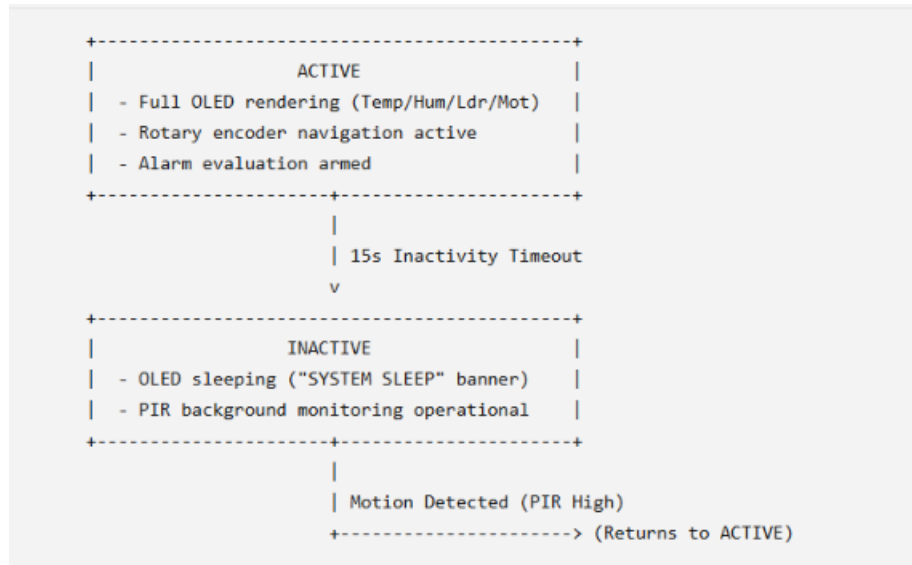
II. Priority Justification

- **Priority 3 (InputTask, MotionTask):** Highest scheduling urgency. Mechanical encoder detents and motion pulses have short physical durations (<50 ms); delayed servicing leads to skipped clicks. Because CPU execution time per wake is minimal (<100 μ s), starvation of lower tasks cannot occur.
- **Priority 2 (SensorTask, AlarmTask, StateTask):** Intermediate urgency. Physical environmental parameters change slowly; however, safety evaluation and state coordination must not be preempted by UI redraws.
- **Priority 1 (DisplayTask):** Lowest urgency. Frame rendering over I2C requires multiple milliseconds of transfer time. Human visual persistence tolerates latencies of 50–100 ms without perceptible lag.

III. State Machine

A two-state finite state machine manages device power modes:

- **ACTIVE State:** OLED displays live sensor metrics; rotary navigation is responsive; temperature alarm monitoring is fully armed.
- **INACTIVE State:** Entered after 15 consecutive seconds without motion. The OLED display is blanked to a static sleep banner to conserve simulated power. Motion detection remains operational in the background.
- **Transition:** Any HIGH pulse on the PIR sensor restores the system to ACTIVE with zero delay.



3. Documentation

a. Critical Code Implementations

1. Drift-Free Periodic Timing (src/sensors.cpp)

Standard `vTaskDelay()` delays relative to function execution, causing period lengthening due to sensor read overhead (≈ 25 ms). `vTaskDelayUntil()` tracks the scheduled tick, preventing drift:

```

TickType_t lastWakeTime = xTaskGetTickCount();
for (;;) {
    float temp = 0.0f, hum = 0.0f;
    if (readDHT22(&temp, &hum)) {
        currentData.temperature = temp;
        currentData.humidity = hum;
    }
    currentData.lightLevel = readLDR();

    // Fan-out to dedicated queues
    xQueueOverwrite(xSensorQueueDisplay, &currentData);
    xQueueOverwrite(xSensorQueueAlarm, &currentData);

    vTaskDelayUntil(&lastWakeTime, pdMS_TO_TICKS(2000));
}

```

2. Mutex-Protected Serial Logging (src/rtos_objects.cpp)

Concurrent writes from multiple tasks to UART standard output were synchronized using xSerialMutex to prevent character-level interleaving:

```
void safeSerialPrintf(const char *format, ...) {
    if (xSerialMutex != NULL && xSemaphoreTake(xSerialMutex, portMAX_DELAY) == pdTRUE) {
        va_list args;
        va_start(args, format);
        vprintf(format, args);
        va_end(args);
        fflush(stdout);
        xSemaphoreGive(xSerialMutex);
    }
}
```

3. Dedicated Fan-Out Queue Configuration (src/rtos_objects.cpp)

Because xQueueReceive() is destructive, a single queue cannot broadcast to multiple tasks. Dedicated consumer queues with length 1 were created for mailbox semantics:

```
xSensorQueueDisplay = xQueueCreate(1, sizeof(SensorData));
xSensorQueueAlarm    = xQueueCreate(1, sizeof(SensorData));
```

4. Results

a. Quantitative Test Data

Automated Unit Tests (Host Environment)

Thirteen-unit tests were executed under the Unity framework to verify core decision logic independently of hardware:

Test ID	Function Under Test	Input Condition	Expected Output	Actual Output	Result
UT-01	evaluateTemperature()	17.9 °C	LOW_TEMPERATURE	LOW_TEMPERATURE	PASS
UT-02	evaluateTemperature()	18.0 °C (Boundary)	NORMAL	NORMAL	PASS

UT-03	evaluateTemperature()	24.5 °C (Mid-range)	NORMAL	NORMAL	PASS
UT-04	evaluateTemperature()	30.0 °C (Boundary)	NORMAL	NORMAL	PASS
UT-05	evaluateTemperature()	30.1 °C	HIGH_TEMPERATURE	HIGH_TEMPERATURE	PASS
UT-06	nextDisplayMode()	TEMPERATURE	HUMIDITY	HUMIDITY	PASS
UT-07	previousDisplayMode()	HUMIDITY	TEMPERATURE	TEMPERATURE	PASS
UT-08	nextDisplayMode()	MOTION (Wraparound)	TEMPERATURE	TEMPERATURE	PASS
UT-09	previousDisplayMode()	TEMPERATURE (Wrap)	MOTION	MOTION	PASS
UT-10	evaluateSystemState()	ACTIVE, no motion, 5s	ACTIVE	ACTIVE	PASS
UT-11	evaluateSystemState()	ACTIVE, no motion, 5s	INACTIVE	INACTIVE	PASS
UT-12	evaluateSystemState()	INACTIVE, no motion, 30s	INACTIVE	INACTIVE	PASS

UT-13	evaluateSystemState()	INACTIVE, motion=true	ACTIVE	ACTIVE	PASS
-------	-----------------------	-----------------------	--------	--------	-------------

Result summary: 13 Tests, 0 Failures, 0 Ignored (100% pass).

Functional Verification (Wokwi Simulation)

The integrated firmware was verified against the ten functional requirements:

Test ID	Requirement	Stimulus	Expected Behavior	Observed Result	Status
FT-01	FR-01	Set DHT22 slider to 41.8 °C	OLED and serial update to 41.8 °C	Serial logged 41.8 °C; OLED updated within 2.0s	PASS
FT-02	FR-02	Set DHT22 humidity to 60.0 %	Humidity screen displays 60.0 %	OLED displayed Hum : 60.0 % accurately	PASS
FT-03	FR-03	Adjust LDR light slider	Light percentage scales 0–100%	ADC mapped smoothly from 0% to 100%	PASS
FT-04	FR-06	Rotate encoder clockwise	Cycles forward with wraparound	Temp → Hum → Light → Motion → Temp confirmed	PASS
FT-05	FR-06	Rotate counter-clockwise	Cycles reverse with wraparound	Temp → Motion → Light → Hum → Temp confirmed	PASS
FT-06	FR-07	Set temp to 41.8 °C (> 30 °C)	Buzzer activates, alert logged	GPIO 23 drove HIGH, buzzer energized, warning logged	PASS

FT-07	FR-07	Return temp to 24.0 °C	Buzzer deactivates, alert clears	GPIO 23 returned LOW, buzzer silenced immediately	PASS
FT-08	FR-04	Trigger PIR motion	System remains / sets ACTIVE	EVENT_MOTION bit set; room activity recognized	PASS
FT-09	FR-09	Inactivity for 15.0 seconds	Screen switches to sleep mode	OLED blanked to "SYSTEM SLEEP [PIR to Wake]"	PASS
FT-10	FR-10	Trigger PIR during INACTIVE	System restores ACTIVE mode	OLED awakened instantly to live metrics display	PASS

Deliberate Fault Experiments

1. **Experiment 1 (Delay Removal):** Removing vTaskDelayUntil() from SensorTask caused Priority 2 CPU monopolization on Core 0. The FreeRTOS IDLE task was starved, triggering the Task Watchdog Timer (TWDT) reset after 5 seconds.
2. **Experiment 2 (Priority Inversion):** Raising DisplayTask from Priority 1 to Priority 4 introduced noticeable lag and dropped clicks on the rotary encoder during I2C rendering. Restoring InputTask to Priority 3 restored instantaneous responsiveness.
3. **Experiment 3 (Mutex Removal):** Removing xSerialMutex from safeSerialPrintf() resulted in corrupted, interleaved text in the terminal when SensorTask and InputTask printed concurrently.

b. Qualitative Analysis

- **What Performed Well:** The rotary encoder navigation was instantaneous. The separation of DisplayTask to Priority 1 completely isolated display write times from sensor acquisitions. The buzzer alarm triggered without latency the moment an out-of-bounds sample arrived.

- **What Required Tuning:** Initial DHT22 bit-banging without open-drain mode and FreeRTOS critical section protection failed to read pulses due to task switching. Adding a 10 kΩ pull-up resistor in diagram.json and enclosing the bit loop in portENTER_CRITICAL() resolved the issue completely.
- **Unique Implementation Detail:** An explicit fan-out queue architecture with mailbox overwrite semantics was implemented rather than relying on global variables, strictly adhering to real-time engineering practices.

5. Conclusions

a. General Reflections & Lessons Learned

The laboratory successfully demonstrated the construction of a deterministic, concurrent embedded system using native ESP-IDF and FreeRTOS on the ESP32. Key concepts mastered include:

- The distinction between vTaskDelay() and vTaskDelayUntil() in eliminating cumulative execution-time drift.
- The necessity of mutexes in protecting non-reentrant hardware peripherals (UART stdout).
- The distinction between Ready, Running, and Blocked task states in priority-based preemptive scheduling.
- The value of separating hardware-independent decision logic into pure functions to facilitate automated host-based unit testing.

b. Issues Faced & Debugging Process

1. **Queue Overwrite Assertion:** Calling xQueueOverwrite() on queues initialized with length 5 caused a FreeRTOS kernel assertion failure (configASSERT(pxQueue->uxLength == 1)). The queue definitions in rtos_objects.cpp were corrected to length 1, satisfying mailbox semantics.
2. **Wokwi Bootloader Offset:** Direct simulation of firmware.bin hung at early boot because ESP-IDF requires the 2nd-stage bootloader (0x1000) and partition table (0x8000). A post-build script (merge_bin.py) was introduced to generate a combined binary (firmware_merged.bin), enabling flawless Wokwi execution.

3. **1-Wire Timing Disruption:** Scheduler preemption disrupted microsecond pulse timing during DHT22 reads. Wrapping the pulse timing sequence in `portENTER_CRITICAL()` ensured atomic capture.

c. Future Improvements

- Implement non-volatile storage (NVS) to allow runtime adjustment of `LOW_TEMPERATURE_LIMIT` and `HIGH_TEMPERATURE_LIMIT` via serial command-line interface.
- Implement hardware interrupt-driven rotary encoder decoding (`gpio_isr_handler_add`) to eliminate periodic polling overhead entirely.
- Calibrate the LDR voltage-divider curve against a laboratory luxmeter to report absolute illuminance in lux.